

Lesson 2: Data Structures & File Handling

Python Essentials — Code the Dream

Game Plan for Today

Section	Time	What We'll Do
Warm-Up	5 min	Reflect on Lesson 1
I Do	10–15 min	Data structures & file handling overview
We Do	15–20 min	Read a CSV into a dictionary together
You Do	5–10 min	Build a mini employee lookup
Wrap-Up	5 min	Assignment preview & Q&A

Warm-Up (5 min)

Reflecting on Week 1

In the chat, share one of these:

- What's one thing that clicked for you in Lesson 1?
- What's one thing you're still fuzzy on?
- Did anyone finish all 10 assignment tasks? What was the hardest one?

I Do: Mutable vs. Immutable

Before we look at data structures, a key concept:

Mutable = can be changed after creation (like clay)

Immutable = cannot be changed after creation (like a ceramic statue)

```
# Mutable: lists
fruits = ["apple", "banana"]
fruits.append("cherry")      # This works!

# Immutable: tuples
colors = ("red", "green")
colors[0] = "yellow"         # TypeError!
```

Quick question: Are strings mutable or immutable in Python?

I Do: The Four Data Structures

Structure	Ordered	Mutable	Duplicates	Key-Value
List	Yes	Yes	Yes	No
Tuple	Yes	No	Yes	No
Dict	No	Yes	Keys: No	Yes
Set	No	Yes	No	No

Each has a purpose — let's look at them one at a time.

I Do: Lists

```
fruits = ["apple", "banana", "cherry"]
fruits.append("orange")      # Add to end
fruits.remove("banana")     # Remove by value
fruits.sort()               # Sort in place
print(fruits[0])            # Access by index: "apple"
```

Key methods: `append()`, `remove()`, `sort()`, `index()`

Lists are **iterable** — you can loop through them:

```
for fruit in fruits:
    print(fruit)
```

I Do: Slicing & Lambdas

Slicing lets you grab a subset of a list:

```
letters = ["A", "B", "C", "D", "E"]
print(letters[1:3])    # ["B", "C"]
print(letters[:2])     # ["A", "B"]
print(letters[-2:])    # ["D", "E"]
```

Lambdas are mini one-line functions:

```
nums = [3, 4, 5]
doubled = list(map(lambda x: x * 2, nums))
# [6, 8, 10]
```

Think of lambdas as a shortcut for simple functions.

I Do: Tuples & Sets

Tuples — ordered, immutable:

```
dimensions = (1920, 1080)
print(dimensions[0]) # 1920
# dimensions[0] = 800 # TypeError!
```

Sets — unordered, no duplicates:

```
numbers = {1, 2, 3, 3, 4}
print(numbers) # {1, 2, 3, 4} – duplicate removed!
```

When to use sets? When you need to remove duplicates or do union/intersection operations.

I Do: Dictionaries

```
person = {"name": "Jazmine", "age": 30}
print(person["name"])           # "Jazmine"
person["email"] = "jazmine@example.com"  # Add new key
```

Key methods:

- `keys()` — all keys
- `values()` — all values
- `items()` — key-value pairs (great for looping!)

```
for key, value in person.items():
    print(f"{key}: {value}")
```

Dicts are everywhere in Python — especially in data work.

I Do: Reading & Writing Files

```
# Reading a text file
with open("example.txt", "r") as file:
    content = file.read()
    print(content)

# Writing to a text file
with open("example.txt", "w") as file:
    file.write("Hello, World!\n")
```

Why `with`? It automatically closes the file — even if an error occurs.

File modes:

- `"r"` — read
- `"w"` — write (overwrites!)
- `"a"` — append (adds to end)

I Do: Reading CSV Files

```
import csv

with open("employees.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)  # Each row is a list of strings
```

The first row is usually headers:

```
["employee_id", "first_name", "last_name", ...]
```

All subsequent rows are data:

```
["1", "Jazmine", "Rivera", ...]
```

What data structure would be good for organizing this?

We Do: Reading a CSV into a Dict

Let's build the `read_employees` function together — this is Task 2 of the assignment!

```
import csv

def read_employees():
    employee_dict = {}
    rows = []
    with open("../csv/employees.csv", "r") as file:
        reader = csv.reader(file)
        for row in reader:
            rows.append(row)
```

What should we do with the first row vs. the rest?

We Do: Separating Headers from Data

```
import csv

def read_employees():
    employee_dict = {}
    rows = []
    with open("../csv/employees.csv", "r") as file:
        reader = csv.reader(file)
        for i, row in enumerate(reader):
            if i == 0:
                employee_dict["fields"] = row
            else:
                rows.append(row)
    employee_dict["rows"] = rows
    return employee_dict
```

Now we have a dict with "fields" and "rows" — just like the assignment wants!

We Do: Using Our Dict

```
employees = read_employees()

# Get column headers
print(employees["fields"])
# ["employee_id", "first_name", "last_name", ...]

# Find the index of a column
name_index = employees["fields"].index("first_name")

# Get a first name from row 0
print(employees["rows"][0][name_index])
```

What function could we write to make this easier?

We Do: Adding Error Handling

What if the file doesn't exist?

```
def read_employees():  
    employee_dict = {}  
    rows = []  
    try:  
        with open("../csv/employees.csv", "r") as file:  
            reader = csv.reader(file)  
            for i, row in enumerate(reader):  
                if i == 0:  
                    employee_dict["fields"] = row  
                else:  
                    rows.append(row)  
            employee_dict["rows"] = rows  
    except Exception as e:  
        print(f"An error occurred: {e}")  
    return employee_dict
```

We Do: Filtering with filter() and Lambdas

The assignment asks you to find an employee by ID. Here's the pattern:

```
def employee_find(employee_id):  
    matches = list(filter(  
        lambda row: int(row[id_column]) == employee_id,  
        employees["rows"]  
    ))  
    return matches
```

What's happening here?

- `filter()` tests each row with the lambda
- The lambda returns `True` if the row's ID matches
- We wrap in `list()` to get a list back

This is directly from Tasks 5 and 6!

You Do: Mini Challenge (5–10 min)

Given this dict structure:

```
employees = read_employees()
```

Try writing one of these:

1. A `column_index(col_name)` function that returns the index of a column header
2. A `first_name(row_num)` function that returns the first name from a given row
3. A `sort_by_last_name()` function using `.sort()` with a lambda `key=`

Hint for sorting:

```
employees["rows"].sort(key=lambda row: row[???])
```

Assignment Preview

This week's assignment has **15 tasks** across two areas:

Tasks	What You'll Build
1	Diary program (file I/O + input)
2–9	Employee CSV reader + lookups
10	Environment variables with <code>os</code>
11	Custom module import
12–15	Minutes CSV: sets, datetime, sorting

Assignment Tips

Start with Task 1 (diary) — it's standalone and builds confidence.

For Tasks 2–9, build one function at a time and run PyTest after each:

```
pytest -v -x assignment2-test.py
```

Key concepts you'll need:

- `csv.reader()` for reading CSV files
- `filter()` with lambdas for searching
- `.sort(key=lambda ...)` for custom sorting
- `with` statements for safe file handling

Tasks 12–15 tip: Rows must be tuples (not lists) to go into a set!

Wrap-Up

Today we covered:

- Four data structures: lists, tuples, dicts, sets
- File I/O with `with open()`
- Reading CSV files into dicts
- `filter()` and lambdas for searching data

Quick check: In the chat, what's one difference between a list and a tuple?

Getting Help This Week

- This assignment is longer than Lesson 1 — **start early!**
- Post questions in **#discussion** on Slack
- Book a **1:1 mentor session** for tricky tasks
- Tasks 12–15 involve sets, datetime, and sorting — these are challenging but rewarding

Next week: More Python Skills — classes, decorators, list comprehensions, and closures!

Happy Coding!

You're building real data skills — keep going!

See you next session!